

Aula 03 - Arrays paralelos para várias contas

Contexto de negócio

Migração de dados: várias contas carregadas em estrutura simples com risco de desalinhamento por índice.

Temos mais dados por conta e mais contas no sistema. Como representar volume no procedural?

Tensão operacional

Com variáveis separadas por pessoa, o código cresce rápido e fica difícil de manter.

Objetivo técnico da entrega

- Arrays paralelos.
- Operação por índice.
- Iteração para listar contas.

Recurso técnico desta aula

Recurso inserido: coleções paralelas alinhadas por posição (**titulares**, **saldos**, **limites**, **ativas**) e regra booleana composta de validação no saque.

Problema que justifica o novo artefato

Até a aula anterior, uma variável por conta funcionava para poucos casos, mas quebra quando o volume cresce.

Problema real:

- precisamos operar várias contas com as mesmas regras;
- copiar variáveis por pessoa aumenta erro humano;
- fica difícil iterar e validar consistência da operação.

Por isso entra o array: ele permite armazenar volume homogêneo e executar algoritmo de repetição com custo previsível.

Artefato explicado: arrays paralelos

Nesta aula, cada posição representa uma conta lógica.

Exemplo de mapeamento por índice:

- índice 0 -> **titulares**[0], **saldos**[0], **limites**[0], **ativas**[0];
- índice 1 -> **titulares**[1], **saldos**[1], **limites**[1], **ativas**[1].

Regra estrutural obrigatória:

- todos os arrays devem ter o mesmo tamanho;
- a mesma posição deve representar a mesma conta em todos os arrays.

Risco previsto:

- se um array for reordenado isoladamente, o algoritmo continua executando, mas com dados cruzados (erro lógico silencioso).

Referência direta do artefato:

- W3Schools Arrays: https://www.w3schools.com/java/java_arrays.asp
- W3Schools For Loop: https://www.w3schools.com/java/java_for_loop.asp

Limitação estrutural deste modelo (questionamento guiado)

Aqui está a fragilidade central deste desenho:

- cada array guarda apenas um pedaço da conta (**titular**, **saldo**, **limite**, **ativa**);
- a conta completa só existe quando juntamos o mesmo índice em todos os arrays.

Isso funciona, mas cobra disciplina alta:

- risco de desalinhamento entre arrays;
- manutenção mais difícil quando surgem novos campos;
- leitura menos natural do algoritmo (dados espalhados em estruturas diferentes).

Questionamento técnico para a turma:

- seria possível representar a conta inteira em uma estrutura única, em vez de espalhar partes em arrays paralelos?
- o que ganhamos e o que perdemos ao manter este formato atual?

Exercício mental importante nesta etapa:

- antes de trocar de abordagem, mapear exatamente onde o modelo atual falha;
- identificar quais erros seriam silenciosos (código roda, mas regra fica errada);
- definir critérios objetivos para decidir quando a estrutura deixa de ser sustentável.

Transição didática para a próxima aula

Arrays resolvem volume inicial, mas cobram disciplina de alinhamento por índice. A próxima etapa consolida o procedural para evidenciar com mais clareza o limite dessa abordagem.

Valor prático entregue

- Permite escalar volume com custo técnico conhecido.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
// Classe temporária de fluxo do programa.
// Neste curso, ela existe para iniciar a execução em Java.
public class AppGestaoContas {

    static void depositar(double[] p_saldos, int p_indice, double p_valor) {
        p_saldos[p_indice] = p_saldos[p_indice] + p_valor;
    }

    static boolean sacar(double[] p_saldos, double[] p_limites, boolean[]
p_ativas, int p_indice, double p_valor) {
        if (!p_ativas[p_indice]) {
            return false;
        }

        double saldoDisponivel = p_saldos[p_indice] + p_limites[p_indice];
        if (p_valor > 0 && p_valor <= saldoDisponivel) {
            p_saldos[p_indice] = p_saldos[p_indice] - p_valor;
            return true;
        }
        return false;
    }

    static void listarContas(String[] p_titulares, double[] p_saldos, double[]
p_limites, boolean[] p_ativas) {
        for (int i = 0; i < p_titulares.length; i++) {
            System.out.println(
                i + " | " + p_titulares[i]
                + " | Saldo: " + p_saldos[i]
                + " | Limite: " + p_limites[i]
                + " | Ativa: " + p_ativas[i]
            );
        }
    }

    public static void main(String[] args) {
        // Aula 03: várias contas com arrays paralelos e índice.
        String[] titulares = {"Ana", "Bruno", "Carla"};
        double[] saldos = {1000.00, 300.00, 700.00};
        double[] limites = {500.00, 100.00, 200.00};
        boolean[] ativas = {true, true, true};

        depositar(saldos, 0, 100.00);
        sacar(saldos, limites, ativas, 1, 350.00);

        listarContas(titulares, saldos, limites, ativas);
    }
}
```

Percorra o código com estes checkpoints de revisão:

- Regra de decisão: mapear condições que aprovam ou negam operações.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Discussão técnica: complexidade da regra booleana

No saque, a decisão é construída por múltiplas condições (**conta ativa**, **valor positivo**, **valor dentro do disponível**).

Mesmo quando essa validação aparece em **if**, a lógica é equivalente a uma expressão booleana composta.

Esse ponto é importante porque:

- regras compostas podem deixar o algoritmo elegante e curto;
- regras compostas também podem virar um bloco difícil de manter;
- qualidade de código depende de equilibrar concisão e clareza.

Diferença de maturidade técnica:

- um código mais fraco apenas “funciona”;
- um código melhor comunica claramente por que aprovou ou negou;
- um código excelente permite evolução de regra sem virar labirinto de condições.

Prática recomendada:

- começar simples;
- quando a regra crescer, quebrar em variáveis intermediárias com nomes semânticos;
- manter o mesmo comportamento observável com melhor legibilidade.

Discussão de contrato de retorno

Quando o método retorna **boolean**, o contrato precisa ser tratado como regra de negócio:

- **true** significa operação aprovada e estado atualizado;
- **false** significa operação negada e estado preservado.

Risco de lógica silenciosa:

- ignorar o **false** no chamador cria fluxo aparentemente correto, mas operacionalmente incompleto.

Responsabilidade de quem chama:

- sempre tratar os dois ramos (**true** e **false**) com mensagens e ações coerentes;
- registrar motivo de negativa quando possível (conta inativa, valor inválido, limite excedido);
- evitar seguir o fluxo como se a operação tivesse ocorrido.

Variações para debate técnico:

1. Em quais cenários **boolean** basta?
2. Quando o time precisa de retorno com motivo da falha?
3. Qual custo de não tratar o **false** em produção?

Paradigma em foco: imperativo, procedural e algoritmo

Leitura de paradigma nesta etapa:

- Imperativo: o programa descreve passo a passo o que a máquina deve executar.
- Procedural: organizamos esse passo a passo em funções reutilizáveis.
- Algoritmo: sequência finita de entrada, processamento, decisão e saída para resolver um problema real.

Por que passar valores nas funções no procedural:

- A função não “conhece” a conta por contexto implícito.
- Tudo que a regra precisa deve chegar por parâmetro.
- O contrato da função fica explícito, testável e auditável.

Risco comum quando isso não fica claro:

- chamar função com parâmetro errado, em ordem errada ou incompleto;
- gerar resultado válido na sintaxe, mas incorreto no negócio.

Ponte para a próxima virada de paradigma:

- No procedural, o estado circula entre funções.
- Na próxima abordagem, vamos discutir alternativas para reduzir esse espalhamento de dados.

Perguntas de decisão

1. O que garante a ligação entre titular e saldo?
2. O que acontece se trocar ordem de apenas um array?
3. Essa estrutura escala com segurança?
4. A regra de saque está no limite ideal de complexidade ou já precisa decomposição?
5. Qual problema operacional o array resolve melhor que variáveis isoladas?

Variações de cenário

1. Reordenar `titulares` e manter `saldos` igual.
2. Tornar `ativas[1] = false` e tentar novo saque.

Exercícios progressivos

1. Criar array `taxas` e incluir na exibição.
2. Implementar transferência entre dois índices.

Desafio de equipe

Criar método `buscarIndicePorTitular(String[] titulares, String nome)` e operar a conta pelo índice encontrado.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.

- Código permanece legível e coerente com o nível da aula.
- A regra de validação de saque continua clara mesmo com evolução de requisitos.

Fechamento executivo

Arrays escalam volume, mas aumentam risco de desalinhamento. Na próxima, vamos consolidar o procedural antes de mudar a modelagem.

Revisão rápida (5 minutos)

1. Regra central: qual condição decide aprovação ou bloqueio na aula?
2. Risco real: qual erro operacional essa implementação reduz?
3. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?

Mini-missão de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteúdo de AppGestaoContas.java no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_arrays.asp
- https://www.w3schools.com/java/java_for_loop.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>